

Outline

From point-wise loss to risk

In L01 we saw the L_2 loss for one point. The **loss function** measures the per-point error:

$$L : \mathcal{Y} \times \mathbb{R}^g \rightarrow \mathbb{R}$$

Common regression losses:

- ▶ L_2 (**quadratic**): $L(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2$
- ▶ L_1 (**absolute**): $L(y, f(\mathbf{x})) = |y - f(\mathbf{x})|$

We want a single number per model, not per point. Two ways:

- ▶ **Theoretical risk**: average over the true (unknown) distribution
- ▶ **Empirical risk**: average over the training data

Theoretical risk

Average loss when using f on data from $\mathbb{P}_{xy}$:

$$\mathcal{R}(f) := \mathbb{E}_{xy}[L(y, f(\mathbf{x}))] = \int L(y, f(\mathbf{x})) d\mathbb{P}_{xy}$$

- ▶ Goal of ML: find $f \in \mathcal{H}$ that minimizes $\mathcal{R}(f)$
- ▶ **Problem:** $\mathbb{P}_{xy}$ is unknown, so we cannot compute the integral
- ▶ Estimating $\mathbb{P}_{xy}$ directly (e.g., kernel density estimation) doesn't scale to high dimensions

Empirical risk

We have n i.i.d. samples — replace the integral with an average:

$$\mathcal{R}_{\text{emp}}(f) = \sum_{i=1}^n L\left(y^{(i)}, f\left(\mathbf{x}^{(i)}\right)\right)$$

Since f is parameterized by θ :

$$\mathcal{R}_{\text{emp}}(\theta) = \sum_{i=1}^n L\left(y^{(i)}, f(\mathbf{x}^{(i)} \mid \theta)\right), \quad \mathcal{R}_{\text{emp}} : \mathbb{R}^d \rightarrow \mathbb{R}$$

One quality score per parameter vector θ . Now the problem is a numerical optimization.

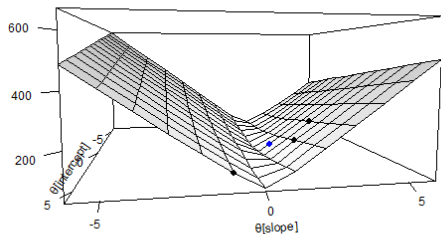
Empirical Risk Minimization (ERM)

Pick the parameter vector with the smallest empirical risk:

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \mathcal{R}_{\text{emp}}(\theta)$$

- ▶ Finite $\mathcal{H}$: tabulate exhaustively (rare)
- ▶ Continuous θ : use **numerical optimization**
- ▶ ERM reduces “learning” to “find the min of a function”

Caveat: optimizing training error too well $\Rightarrow$ overfitting (L03–L04).



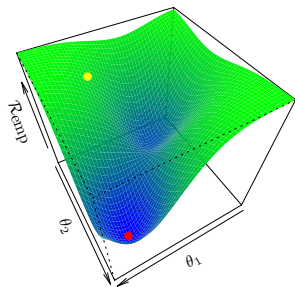
The error surface

Treat $\mathcal{R}_{\text{emp}}(\boldsymbol{\theta})$ as a function over the parameter space.

- ▶ Find $\hat{\boldsymbol{\theta}}$ by traversing the surface downhill
- ▶ For continuous $\boldsymbol{\theta}$: *local* search from a start point
- ▶ Best optimizer depends on problem structure

Local vs global:

- ▶ **Global:** $\mathcal{R}_{\text{emp}}(\hat{\boldsymbol{\theta}}) \leq \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) \forall \boldsymbol{\theta}$
- ▶ **Local:** holds only in a neighborhood
- ▶ Every global min is local. Finding a local min is much easier.



Stationary points

If $\mathcal{R}_{\text{emp}}$ is continuously differentiable, a **sufficient condition** for a local minimum:

► Gradient is zero: $\frac{d\mathcal{R}_{\text{emp}}}{d\theta}(\hat{\theta}) = 0$

► Hessian at $\hat{\theta}$ is positive definite

► *Negative* gradient points in the direction of steepest descent

► Hessian measures local curvature

Gradient descent

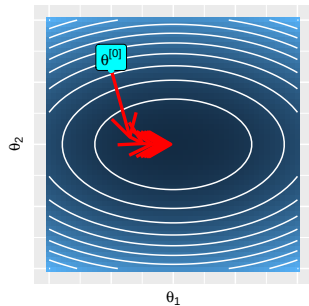
Start from some $\theta^{[0]}$. Iteratively move in the direction of steepest descent:

$$\theta^{[t+1]} = \theta^{[t]} - \alpha \frac{d\mathcal{R}_{\text{emp}}}{d\theta}(\theta^{[t]})$$

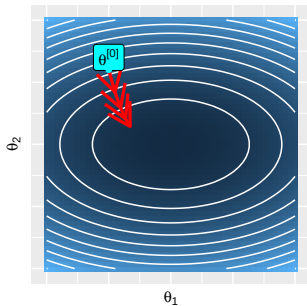
- ▶ α : **step size / learning rate**
- ▶ Negative gradient is direction of steepest local decrease
- ▶ Magnitude $\| -\nabla \mathcal{R}_{\text{emp}} \|$ tells us only about *local* steepness — we have no guarantee against overshooting

Learning rate sensitivity

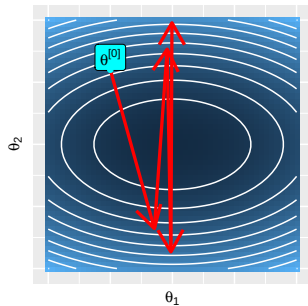
Step size α is the dominant hyperparameter.



good convergence (α_1)



slow ($\alpha_2 < \alpha_1$)



no convergence ($\alpha_3 > \alpha_1$)

Practical: constant α tuned by trial; decay schedules $\alpha_t = \alpha_0 / (1 + \gamma t)$; modern adaptive optimizers (Adam — see L18).

Beyond vanilla GD

- ▶ A few models (e.g., linear regression with L_2) admit a **closed-form** solution. Most don't.
- ▶ GD is a **first-order** method. **Second-order** methods (Newton–Raphson) use the Hessian for faster convergence but cost more per step.
- ▶ Many improvements: **momentum**, **Adam**, **RMSprop** — adaptive learning rates, escape saddle points, mimic curvature without computing the Hessian.
- ▶ **Stochastic gradient descent (SGD)**: compute the gradient on a small random batch, not the full dataset. Much more efficient on large data.

Why consider L_1 ?

- ▶ L_2 minimizes squared residuals — wouldn't *absolute* residuals seem more natural?
- ▶ L_1 loss / **Least Absolute Deviation (LAD)**:

$$L(y, f(\mathbf{x})) = |y - f(\mathbf{x})|$$

- ▶ Also called **median regression** — at the optimum the model predicts the conditional median of y , while L_2 predicts the conditional mean.

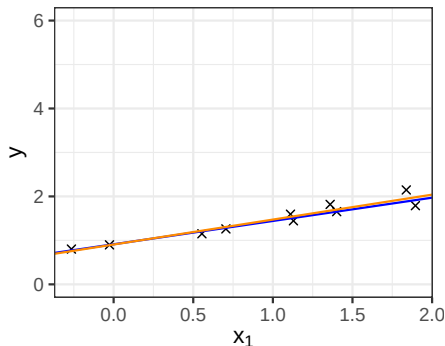
L_1 vs L_2 : optimization difficulty

Loss surfaces look very different:

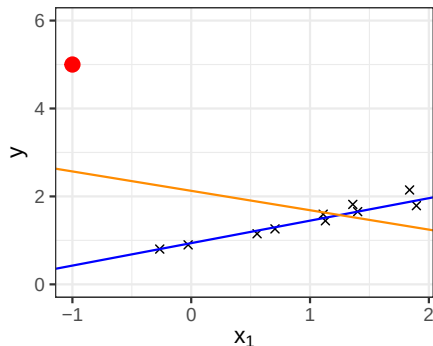
- ▶ L_2 : smooth, convex, differentiable everywhere $\Rightarrow$ closed-form OLS
- ▶ L_1 : convex, but **not differentiable** at $y = f(\mathbf{x})$
 - ▶ No closed-form solution
 - ▶ Use subgradient methods or linear programming
 - ▶ In practice: slower to fit than OLS

L_1 vs L_2 : robustness

- ▶ L_2 is quadratic in residuals $\Rightarrow$ outlying points carry massive weight
- ▶ Example: $3\times$ residual $\Rightarrow 9\times$ loss contribution
- ▶ L_1 scales linearly $\Rightarrow$ **more robust** to outliers



 absolute  quadratic



 absolute  quadratic

L_1 vs L_2 : when to use which

Prefer L_1 when:

- ▶ Data has heavy-tailed noise or suspected outliers
- ▶ You care about the median behavior, not the mean

Prefer L_2 when:

- ▶ Closed-form solution wanted
- ▶ Noise is roughly Gaussian
- ▶ Speed matters (e.g., inside a CV loop)

Cost in practice

Toy weather problem (predict mean temperature):

- ▶ Coefficients fitted under L_1 and L_2 differ a little, not a lot
- ▶ L_1 achieves slightly lower total L_1 -loss; L_2 slightly lower L_2 -loss
- ▶ L_1 runs noticeably **slower** as dataset size grows

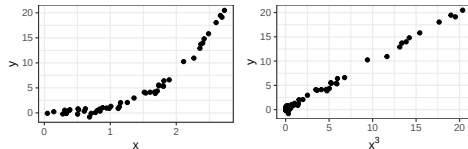
Takeaway: choice of loss matters for *robustness*, more than for typical-case parameter estimates on clean data.

Linear is more flexible than it looks

- ▶ “Linear regression” is linear *in the parameters*, not necessarily in features
- ▶ Generalization with basis functions ϕ_j :

$$f(\mathbf{x}) = \theta_0 + \sum_{j=1}^p \theta_j \phi_j(x_j)$$

- ▶ Vanilla LM: $\phi_j(x) = x$ for all j
- ▶ Solution via normal equations, just with the transformed $\mathbf{X}$



y vs x looks nonlinear, but y vs x^3 looks pretty linear.

Is $f(x) = \theta_0 + \theta_1 x^2$ really “linear”?

- ▶ **Not** in the input x : $f(ax + bx_*) \neq af(x) + bf(x_*)$
- ▶ But **yes** in the parameters θ :

$$af(x, \theta) + bf(x, \theta^*) = f(x, a\theta + b\theta^*)$$

This is the key property that lets us re-use OLS, gradient descent, all the linear-model machinery, on highly nonlinear functions.

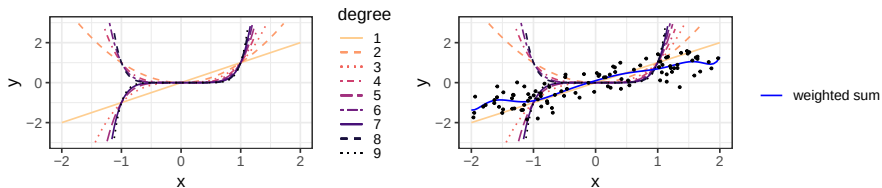
Polynomial basis functions

Map x_j to its first d powers:

$$\phi^{(d)} : \mathbb{R} \rightarrow \mathbb{R}^d, \quad x \mapsto (x, x^2, \dots, x^d)$$

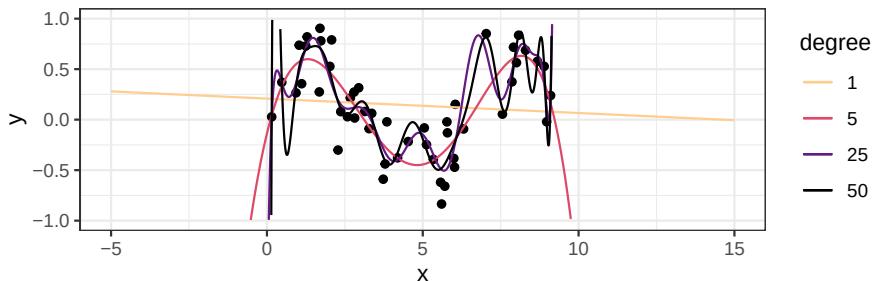
Univariate model:

$$f(\mathbf{x}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d$$



Higher $d \Rightarrow$ more flexible

- ▶ Larger d = richer hypothesis space = higher **capacity**
- ▶ Tempting: just let $d \rightarrow \infty$? **No.**
- ▶ Data has random **noise** that is not part of the true relationship
- ▶ Too much capacity fits the noise $\Rightarrow$ **overfitting** $\Rightarrow$ poor generalization (L04)
- ▶ Also: high-degree polynomials oscillate at edges (**Runge's phenomenon**)



Worked example: bike rentals

OpenML dailybike: predict daily rentals from weather features.

$$f(\mathbf{x}) = \sum_{k=1}^d \theta_{\text{temp},k} x_{\text{temp}}^k + \theta_{\text{season}} x_{\text{season}} + \theta_{\text{hum}} x_{\text{hum}}$$

Cross-validated test error (more on CV in L05):

- ▶ $d = 1$ (linear in temperature): test error 121.9
- ▶ $d = 3$ (cubic): test error 117.6 $\Leftarrow$ best
- ▶ Going higher: test error rises again (overfit)

Conclusion: a small dose of flexibility helps. Too much hurts. **Choosing d is a tuning problem** (L05).

What we covered

1. Defined loss, theoretical risk, empirical risk
2. ERM: training as numerical optimization of $\mathcal{R}_{\text{emp}}(\boldsymbol{\theta})$
3. Gradient descent: iterative downhill, sensitive to α
4. L_1 vs L_2 tradeoffs: optimization cost vs robustness
5. Polynomial regression: linear-in-parameters $\neq$ linear-in-features

Next (L03): when ERM picks too-complex models, what do we do? $\rightarrow$ regularization.